Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A Constituent Unit of Yenepoya (Deemed to be University), Balmatta, Mangalore

Department of Computer Science

LOW-LEVEL DESIGN (LLD)

on

"Identity Security Framework for Agriculture"

Student Name: Mohammed Thwaha A T P

Register Number: 23BCCE137

Programme: BCA – Cyber Security, Ethical Hacking & Cloud Computing with IBM

Industry Mentor: Shashank Guided by: Ms.Vijayalaxmi

Table of Contents

1. Introduction..... 3
1.1. Scope..... 3
1.2. Intended Audience..... 3
2. System Design..... 3
2.1. Application Design..... 3
2.2. Process Flow..... 4
2.3. ComponentsDesign..... 5
2.4. Key Design Desicions..... 5
2.5. API Catalogue..... 6
3. Data Design.....7
3.1. Database Tables..... 7
3.2. Data Access..... 9
3.3. Data Retention..... 9
4. Interfaces..... 9
5. Session Management..... 9
6. Non-Functional Requirements..... 10
6.1 Security……..……10
6.2 Performance……...10
7. References.....11

1. INTRODUCTION

This Low-Level Design document explains how the Identity Security Framework for Agriculture is implemented in code. It covers file structure, database tables, Flask route functions, validation rules, security controls, session keys, and interface behavior.

1.1 Scope

The scope includes app.py, database.db, the HTML templates, static uploads, authentication flows, seller approval, product upload, dashboard rendering, and local SQLite persistence.

1.2 Intended Audience

This document is written for project evaluators, the internal guide, the industry mentor, and developers who may maintain or extend the project.

2. SYSTEM DESIGN

2.1 Application Design

The project is implemented as a single Flask application. The main file app.py initializes the app, creates SQLite tables, defines route functions, validates inputs, handles email OTP, uploads product images, and renders templates. SQLite stores data in database.db. HTML templates are stored in the templates folder and product images are stored in static/uploads.

Layer/File	What it does
app.py	Defines Flask app, database initialization, route handlers, authentication, OTP, seller approval, and product upload logic.
database.db	SQLite database containing users, seller_requests, products, and activity_logs tables.
templates/index.html	Landing page with login and signup navigation.
templates/buyer_signup.html	Buyer registration form.
templates/buyer_login.html	Buyer login form using email and password.
templates/staff_login.html	Seller/Admin login form using user ID and password.
templates/verify_login.html	OTP verification form.
templates/dashboard.html	Buyer product listing dashboard.
templates/seller_dashboard.html	Seller add-product form and product grid.
templates/admin_dashboard.html	Admin stats and pending seller approval table.
templates/manage_users.html	Admin user listing page.
static/uploads	Stores uploaded product images such as tomato and orange images.

Technology	Implementation Use
Python	Main programming language for backend logic.
Flask	HTTP routing, request handling, rendering templates, and sessions.
SQLite	Lightweight local relational database.
Werkzeug	Password hashing and secure uploaded filename handling.
smtplib	SMTP email delivery for OTP and seller approval messages.
Jinja2	Template rendering for dynamic dashboards.

2.2 Process Flow

- Buyer Signup: /buyer_signup receives form data, validates password strength, hashes password, inserts buyer into users, then redirects to /buyer_login.
- Buyer Login: /buyer_login fetches buyer by email and role, checks password hash, generates OTP, stores temp_user and otp in session, sends email, then redirects to /verify_login.
- Seller Request: /seller_request stores public seller application details in seller_requests and redirects to /request_sent.
- Admin Approval: /approve_seller/<id> fetches the request, creates a SELL#### user ID, stores temporary hashed password, deletes the request, and emails credentials.
- Staff Login: /staff_login authenticates seller/admin by user_id, sends OTP, and redirects to verification.
- OTP Verification: /verify_login compares the entered OTP with session['otp'], creates user_id and role session keys, and redirects based on role.
- Force Reset: /force_reset updates a seller temporary password and sets is_temp_password to 0.
- Product Upload: /add_product sanitizes image filename, saves image to static/uploads, inserts product row, and redirects to seller dashboard.

```
seller_request -> seller_requests table -> admin_dashboard
                  |                        |
                  |                        v
          +----------------------------- approve_seller(id)
                                           |
                                           v
                                users table creates seller
                                           |
                                           v
                              email user_id/temp password
```

Figure 1 - Seller Approval Flow

2.3 Components

- Authentication: buyer_login, staff_login, verify_login, force_reset, logout.
- Registration and Onboarding: buyer_signup, seller_request, request_sent, approve_seller.
- Buyer Module: dashboard displays all products from the products table.
- Seller Module: seller_dashboard displays products belonging to session['user_id']; add_product inserts new listings.
- Admin Module: admin_dashboard shows seller requests and platform counts; manage_users lists all accounts.
- Database Initialization: init_db creates tables and seeds the default admin account.
- Email Module: send_email uses SMTP, STARTTLS, and Gmail credentials to send OTP/approval messages.
- Upload Module: secure_filename and static/uploads handle product image storage.

2.4 Key Design Decisions

- Single-file backend keeps the academic project easy to inspect and run.
- SQLite avoids external database installation and keeps data in one local file.
- Separate buyer and staff login pages reduce confusion between email login and user ID login.
- OTP verification demonstrates two-factor authentication in a simple way.
- Temporary seller passwords force sellers to reset credentials after admin approval.
- Product image filenames are stored in the database rather than binary image data.
- POST-Redirect flow is used after successful form submissions to reduce duplicate actions on refresh.

2.5 API Catalogue

URL	Method	Access / Purpose
/	GET	Public - render landing page.
/buyer_signup	GET, POST	Public - buyer registration and password strength validation.
/seller_request	GET, POST	Public - seller application form.
/request_sent	GET	Public - seller request confirmation page.
/buyer_login	GET, POST	Public - buyer credential check and OTP generation.
/staff_login	GET, POST	Public - seller/admin credential check and OTP generation.
/verify_login	GET, POST	Session - OTP comparison and role-based redirect.
/force_reset	GET, POST	Logged-in temporary seller - replace temporary password.
/approve_seller/<id>	GET	Admin workflow - approve seller request and create account.
/dashboard	GET	Buyer - show all products.
/seller_dashboard	GET	Seller - show add product form and seller products.
/admin_dashboard	GET	Admin - platform stats and pending seller requests.
/add_product	POST	Seller - upload product image and insert product.
/manage_users	GET	Admin - list all users.
/activity_logs	GET	Admin - reserved page for audit logs.
/logout	GET	Logged-in users - clear session and return home.

3. DATA DESIGN

3.1 Database Tables

All application tables are created automatically by init_db(). The database file is database.db. The sqlite_sequence table is created by SQLite internally for AUTOINCREMENT values and is not part of the application design.

Table	Columns
users	id INTEGER PRIMARY KEY AUTOINCREMENT; name TEXT; email TEXT; user_id TEXT UNIQUE; password TEXT; role TEXT; is_temp_password INTEGER DEFAULT 0.
seller_requests	id INTEGER PRIMARY KEY AUTOINCREMENT; name TEXT; email TEXT; phone TEXT; business_name TEXT; status TEXT DEFAULT 'pending'.
activity_logs	id INTEGER PRIMARY KEY AUTOINCREMENT; user_id TEXT; action TEXT; timestamp TEXT.
products	id INTEGER PRIMARY KEY AUTOINCREMENT; name TEXT; price REAL; description TEXT; image TEXT; seller_id TEXT.

Table	What it stores
users	All registered identities: default admin, buyers, and approved sellers. The password column stores a generated hash, not the plain password.
seller_requests	Public applications submitted by farmers or agri sellers. Admin reviews and approves these requests.
products	Agricultural products listed by sellers. Image is the uploaded filename stored in static/uploads.
activity_logs	Audit table designed to store user activity such as login, logout, approval, and product upload events.

Important users table fields

- user_id is used as the login identifier for sellers and admin. For buyers, email is also stored as user_id.
- role controls whether the user acts as buyer, seller, or admin.
- is_temp_password is set to 1 for newly approved sellers and reset to 0 after password change.
- password contains the output of generate_password_hash().

3.2 Data Access

- sqlite3.connect('database.db') opens a database connection inside each route.
- cursor.execute() uses ? placeholders for values submitted by users.
- conn.commit() is called after INSERT, UPDATE, and DELETE operations.
- conn.close() is called after database work to release the connection.
- Products are fetched using SELECT queries and passed to templates as tuple lists.

Operation	SQL Pattern
Create buyer	INSERT INTO users (name, email, user_id, password, role) VALUES (?, ?, ?, ?, ?)
Find buyer	SELECT * FROM users WHERE email=? AND role='buyer'
Find staff	SELECT * FROM users WHERE user_id=?
Approve seller	INSERT INTO users (...) followed by DELETE FROM seller_requests WHERE id=?
List products	SELECT * FROM products
Seller products	SELECT name, price, image FROM products WHERE seller_id=?

3.3 Data Retention

The current local version does not automatically delete historical records. Records remain in database.db until removed manually. In production, activity logs should be archived after 90 days, seller requests should be retained for approval history, and product image files should be deleted when a product is removed.

4. INTERFACES

The application has only one user-facing interface: a browser-based web interface running at http://localhost:5000. All forms submit traditional HTTP GET and POST requests. There is no mobile app, REST API, or webhook integration.

Template	Route	Main Fields / Display
index.html	/	Navigation to buyer signup/login and seller request/staff login.
buyer_signup.html	/buyer_signup	name, email, phone, password.
buyer_login.html	/buyer_login	email, password.
staff_login.html	/staff_login	user_id, password.
verify_login.html	/verify_login	OTP.
force_reset.html	/force_reset	new password.
seller_request.html	/seller_request	name, email, phone, business.
dashboard.html	/dashboard	product cards with image, name, and price.
seller_dashboard.html	/seller_dashboard	add product form and seller product cards.
admin_dashboard.html	/admin_dashboard	stats, seller requests table, approve links.
manage_users.html	/manage_users	all user accounts.
activity_logs.html	/activity_logs	audit log page layout.

5. SESSION MANAGEMENT

Session state is stored using Flask's signed session cookie. The application uses short-lived temporary session keys during OTP login and longer-lived keys after OTP verification.

Key	Created In	Deleted In / End	Purpose
temp_user	buyer_login/staff_login	After login flow or logout	Stores selected user tuple before OTP verification.
otp	buyer_login/staff_login	After login flow or logout	Stores generated OTP for comparison.
user_id	verify_login	logout	Stores authenticated user identifier.
role	verify_login	logout	Stores buyer/seller/admin role for dashboard selection.

- The login flow does not create a full authenticated session until OTP succeeds.
- force_reset uses session['user_id'] to update the password for the currently logged-in seller.
- logout calls session.clear() and redirects to the home page.

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Security

- Password hashing: generate_password_hash() stores hashed passwords.
- Password validation: minimum 8 characters, uppercase, lowercase, digit, and special character.
- Two-factor authentication: OTP sent by email after password verification.
- Role-based separation: users have buyer, seller, or admin roles.
- File upload safety: secure_filename() sanitizes uploaded image names.
- Parameterized SQL: route queries use placeholders for submitted values.
- Session cleanup: logout clears all active session values.

6.2 Performance

- The application is small and runs quickly on localhost.
- SQLite is suitable for the academic dataset and does not require a separate database server.
- Server-side rendering keeps page logic simple and fast.
- Static product images are served directly from the static/uploads folder.
- SMTP calls are the slowest step and may take a few seconds during OTP delivery.

7. REFERENCES

- Flask Documentation - https://flask.palletsprojects.com/
- Flask-Login Documentation - https://flask-login.readthedocs.io/
- Werkzeug Security Utilities - https://werkzeug.palletsprojects.com/
- Python sqlite3 Documentation - https://docs.python.org/3/library/sqlite3.html
- Python smtplib Documentation - https://docs.python.org/3/library/smtplib.html
- OWASP Authentication Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Session Management Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html